bias_initialization.py

`src/moe_pruning/pruning/bias_initialization.py · 138 lines · module moe_pruning.pruning.bias_initialization`

What it does

Computes the starting value of each bias vector from the collected statistics: the weighted mean of the expert's outputs with weight s_t * g^2 (EASY-EP token contribution times squared gate), which minimises the contribution-weighted squared error of the layer output.

How it works

- Uses only running sums, so no per-token data is stored.
- Fallbacks when the weight sum is zero: gate-weighted mean, then plain mean; each fallback is logged and flagged.
- An expert never seen in calibration raises `UnobservedExpertError` unless `PRUNE_UNOBSERVED_POLICY=allow` (then a zero vector, flagged).
- Reports how much of the expert's output energy a constant leaves unexplained (relative residuals); values near 1 mean a constant is a poor stand-in.

Defined in this file

Extracted from the source code (classes, public functions and methods).

Name	Description
`class UnobservedExpertError`	Raised when an expert never seen in calibration would need a bias.
`weighted_mean(weighted_sum, weight_total)`	sum_t w_t x_t / sum_t w_t, or None when the denominator is (numerically) zero.
`weighted_residual(sq_sum, weighted_sum, weight_total, b)`	sum_t w_t ||x_t - b||^2 from running sums (clamped at 0 for rounding).
`class BiasInitResult`	Method used, fallback flag, token count, bias norm and residual diagnostics.
`.to_dict()`	Convert to a plain dictionary for JSON reports.
`initialize_bias(stats, expert_id, allow_unobserved)`	Compute b_e (float64) for one expert from accumulated statistics.

Connections

Uses: `moe_pruning.scoring.statistics`

Used by: `scripts/replace_experts.py`